SciMARS: Multi-Agent Recursive Self-Improvement for Closed-Loop Scientific Discovery

Weimin Wu[†], **Han Liu**[†,§]

[†] Center for Foundation Models and Generative AI, Northwestern University
Department of Computer Science, Northwestern University
[§] Department of Statistics and Data Science, Northwestern University
wwm@u.northwestern.edu, hanliu@northwestern.edu

We propose SciMARS, a framework for recursive task-local adaptation of multi-agent harnesses in closed-loop scientific discovery. Multi-agent systems coordinate analysis, experimental design, and verification as scientific evidence evolves. However, two problems limit their effectiveness: (i) without shared memory, agents lose observations and repeat work; (ii) fixed role procedures, handoffs, and memory policies perpetuate mistakes despite retained evidence. We address these problems with a shared evidence graph and separate component editors. SciMARS adapts roles and coordination; SciMARS + Memory adds a third editor for how agents record, retrieve, and receive evidence. Candidate probes guide composition and selection of the next harness. We evaluate scientific discovery through experimental-data replay and a digital twin. SciMARS + Memory resolves 12/18 GPT-6-Astra and 2/18 GLM-5.3 attempts on Autonomous Science Bench, compared with 8/18 and 1/18 for the base multi-agent team. On ScienceAgentBench and AppWorld, it solves 14/38 and 11/20 tasks, compared with 11/38 and 5/20 for the base team. These results support combining shared memory with harness adaptation across scientific and interactive tasks.
Project page: https://github.com/WeiminWu2000/SciMARS

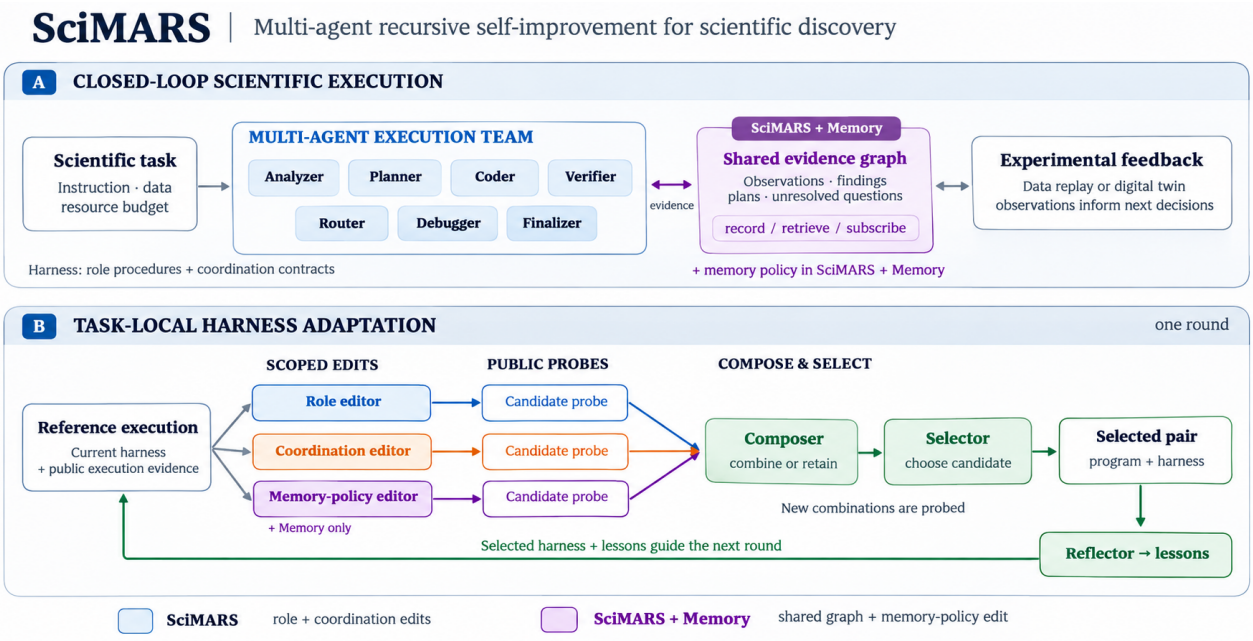


Figure 1 | SciMARS overview. (A) Execution roles use experimental feedback and optional shared memory. (B) Role and coordination editors propose candidates; SciMARS + Memory adds a memory-policy editor. Public probes guide composition and selection. The selected harness and lessons guide the next round.

1. Introduction

We propose SciMARS, a framework for multi-agent recursive self-improvement through task-local harness adaptation in closed-loop scientific discovery, linking in silico analysis with simulated experimental validation. Scientific discovery advances through cycles of hypothesis generation, experimental design, analysis, and validation. Closed-loop systems connect computational decisions with experimental feedback, so each new observation can inform the next experiment. Multi-agent systems divide this work among specialized roles (Wu et al., 2023; Hong et al., 2024). As evidence changes, these roles must preserve useful findings, communicate constraints, and revise their next steps. Two problems limit this process: evidence becomes isolated across agents, and fixed execution rules repeat mistakes despite access to that evidence.

The first problem is the lack of shared memory. An observation or failed check may remain in one agent's history without reaching another agent whose decision depends on it. A planner may repeat a rejected experiment, or a coder may miss a constraint that the verifier has identified. We address this gap with a shared evidence graph inspired by Mycelium's active context graphs (Choudhury et al., 2026). The graph records observations, findings, plans, and unresolved questions with provenance. Retrieval and subscriptions route these records to roles beyond the current handoff.

The second problem is a fixed execution harness. Retaining a failed check does not ensure that a planner acts on it, a coder changes its procedure, or a verifier retrieves the right record. Role instructions may omit a required check, handoffs may drop its result, and memory policies may retrieve irrelevant notes. The system must adapt how agents work, exchange information, and use retained evidence. Memory contents and policy serve distinct purposes: records preserve observations; policies govern their use in later decisions.

We introduce **SciMARS**, a unified framework for task-local harness adaptation in closed-loop scientific discovery. Recent methods improve agent harnesses and workflows from execution feedback (Xu et al., 2026; Lee et al., 2026; Feng et al., 2026). SciMARS decomposes this adaptation into separate role and coordination proposals. Each editor starts from the same reference execution and proposes a change within its own scope. The controller probes the candidates, composes a successor from their evidence, and selects a harness for the next round. Retained lessons guide later revisions within the same task. This is recursion over the execution harness; the meta-level optimizer does not rewrite itself. Figure 1 illustrates the role–coordination loop.

We study two adaptation settings. **SciMARS** updates role procedures and coordination contracts. **SciMARS + Memory** adds the shared graph and a third independent editor for memory policy. This editor changes how roles record, retrieve, and receive evidence while preserving the graph's provenance rules. Candidate probes let the composer assess each proposed component before combining changes; a new combination receives its own probe. A fixed-policy Multi-Agent + Memory control distinguishes access to shared evidence from adaptation of its use. All adaptation decisions use public execution evidence, with official benchmark scores reserved for final evaluation.

We evaluate the discovery interface through experimental-data replay and a digital twin in Autonomous Science Bench (Wen and Liu, 2026). Its six campaigns span biology, chemistry, and materials science. This protocol tests decisions under simulated experimental feedback rather than new physical wet-lab execution. ScienceAgentBench (Chen et al., 2025) evaluates scientific programming, and AppWorld (Trivedi et al., 2024) evaluates interactive execution across tools and application state. We compare the base multi-agent team, fixed memory, integrated workflow search, SciMARS, and SciMARS + Memory. The study covers GLM-5.3 variants (Z.ai, 2026) and a GPT-6-Astra discovery cohort, with the base team as the common reference for the headline results.

Our contributions are threefold:

(i) **A unified framework for task-local adaptation for closed-loop scientific discovery.** We use execution evidence to adapt multi-agent systems for scientific analysis and experimental design, evaluating the interface through experimental-data replay and a digital twin. A shared evidence graph carries observations, checks, and unresolved questions across roles and task steps.

(ii) **Separate adaptation of roles, coordination, and memory use.** SciMARS assigns role procedures and coordination contracts to distinct editors. SciMARS + Memory adds an independent editor for memory policy. We probe the proposals and use their evidence to compose and select a successor, rather than revising all components in one integrated proposal.

(iii) **Experimental results across scientific and interactive tasks.** SciMARS + Memory resolves 12/18 GPT-6-Astra and 2/18 GLM-5.3 attempts on Autonomous Science Bench, compared with 8/18 and 1/18 for the base multi-agent team. It solves 14/38 ScienceAgentBench tasks and 11/20 AppWorld tasks, compared with 11/38 and 5/20 for the base team. Overall, these results show that combining shared memory with SciMARS improves multi-agent performance.

2. Related Work

We review related work on multi-agent systems, agentic scientific discovery, and recursive self-improvement to position our approach to task-local harness adaptation.

Multi-agent systems. Language-model agents support complex workflows through reasoning, tool use, and execution feedback (Lee et al., 2026; Nam et al., 2026). Multi-agent frameworks organize these agents into specialized roles with explicit interaction patterns. AutoGen (Wu et al., 2023) coordinates agents through conversation, while MetaGPT (Hong et al., 2024) assigns role-specific procedures. More recent methods adapt the workflows connecting these roles: AFlow (Zhang et al., 2025b) uses execution feedback to optimize workflows, and MASS (Lee et al., 2026) searches over roles and information routes. DS-STAR (Nam et al., 2026) provides a data-science agent pipeline and serves as the basis for our execution harness. We introduce the shared memory to multi-agent team (Choudhury et al., 2026) and extend this team with task-local adaptation.

Agentic systems for scientific discovery. AI models support scientific applications ranging from genomics and single-cell analysis (Wang et al., 2025, 2026; Wu et al., 2025a; ElSheikh et al., 2026; Wu et al., 2025b; Zhou et al., 2024). Scientific agents use language models to address different parts of scientific discovery. Our framework targets the coordination of computational design and experimental feedback; we study this interface through data replay and a digital twin. ScienceAgentBench (Chen et al., 2025) evaluates agents on data-driven scientific workflows, while Autonomous Science Bench (Wen and Liu, 2026) evaluates budgeted experimental-design campaigns using experimental-data replay and a digital twin. Recent work such as ADAPT for Science (Feng et al., 2026) evolves scientific optimization procedures from simulation feedback. We test computational decisions under simulated wet-lab experimental feedback.

Recursive self-improvement. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) use feedback or retained lessons to improve subsequent attempts, while program-improvement methods such as STOP (Zelikman et al., 2023) and the Darwin Gödel Machine (Zhang et al., 2025a) revise programs or agent implementations. Recent frameworks extend this idea to agent harnesses and multi-agent workflows: Harness^2 (Xu et al., 2026) uses a task-level propose–probe–compose loop and a cross-task experience mechanism, while MASS (Lee et al., 2026) combines workflow search with model fine-tuning. These methods act on different adaptation surfaces, from feedback and workflow structure to agent code. SciMARS focuses on task-local execution-harness adaptation. It separates role and coordination edits; its memory extension adds a separate policy editor without changing the graph service or the optimizer.

3. SciMARS

SciMARS adapts a team through execution, scoped proposals, candidate probes, composition, selection, and reflection. It builds on the propose–probe–compose principle of Harness[2] (Xu et al., 2026). We first define the team and shared memory, then describe one adaptation loop for two settings: SciMARS edits roles and coordination; SciMARS + Memory also edits memory. Table 1 summarizes the loop.

3.1. Problem and accessible evidence

A task provides an instruction x, public inputs D, and a resource allowance B. An execution harness H governs the team. The executor E produces a program or action trajectory p, an execution trace τ, output artifacts or environment state a, and resource use c:

$$o = (p, \tau, a, c) = E(H; x, D). \tag{1}$$

The optimizer seeks a harness that helps the team satisfy the task within B. It observes public inputs, tool responses, candidate programs, and generated artifacts. It cannot query the official evaluator V during revision. A host process calls V after final commitment. A runnable program, completed tool call, or recorded finding does not establish task correctness. Public probes guide revision; official evaluation measures the final outcome.

3.2. Team harness and editable components

SciMARS represents the harness at round t as

$$H_t = (P_t, C_t), \qquad P_t = \{p_{t,r} : r \in \mathcal{R}\}. \tag{2}$$

The role set $\mathcal{R}$ contains an analyzer, planner, coder, verifier, router, debugger, and finalizer. They inspect inputs, plan steps, implement them, check results, route work, repair failures, and prepare deliverables. These are logical responsibilities. They may use the same model, and an adapter may group responsibilities into fewer execution agents.

The coordination component C_t contains routing flags and contracts for existing handoffs. A contract specifies a version, shared fields, a sender instruction, and a receiver instruction. A planner–coder contract can require observed array dimensions in the handoff. A role procedure can instead require the coder to check its inputs. The former changes handoffs; the latter changes role procedures.

The seven-role implementation exposes eight interfaces: analyzer–planner, planner–coder, coder–verifier, verifier–router, router–planner, verifier–planner, verifier–finalizer, and coder–finalizer. Available fields include the question, filenames, file descriptions, plans, code, execution output, verification, and routing state. An editor cannot add a field that the executor does not provide. Contract versions track revisions, not quality.

A separate meta-level optimizer invokes the editors, composer, selector, and reflector. These are not executions of the seven-role team. The optimizer's prompts, validators, selection algorithm, tools, and official evaluator remain outside the editable surface. The controller retains the current harness, task-local lessons L_t, and traces T_t. Accepted edits govern later rounds of the same task; they do not become a shared learned harness for later evaluation tasks.

3.3. Shared memory: evidence contents and usage policy

Our shared memory draws on Mycelium's active context-graph design (Choudhury et al., 2026). It lets a role retrieve evidence recorded by another role beyond the current handoff. We distinguish

graph contents G_t from a memory policy M_t. The graph holds observations, interpretations, findings, hypotheses, plans, and unresolved questions. The policy determines what roles record and which records reach them. New findings change G_t; retrieval rules belong to M_t.

Records and provenance. The graph uses six namespaces: context, work, decisions, status, procedures, and log. Records have keys, types, authors, and versions. Typed edges express support, contradiction, derivation, and dependency. Agent notes link to existing records; source observations can name public files or execution events. Dataset and observation records cannot be overwritten. Host validation enforces the schema and provenance links. These checks preserve an evidence trail but do not certify a claim's truth.

Writing and retrieval. The host records public context and execution observations. Roles add grounded notes through the adapter's memory interface. A handoff supplies policy-filtered records and a feed of new records from other roles. Retrieval uses node types, namespaces, a query hint, and a record limit. Lexical relevance ranks records, with recency breaking ties. A verifier can link a failed shape check to its execution. The planner retrieves this finding to revise its plan.

The extended harness is

$$H_t = (P_t, C_t, M_t), \qquad M_t = \{(w_{t,r}, q_{t,r}, s_{t,r}) : r \in \mathcal{R}\}, \tag{3}$$

where $w_{t,r}$ specifies recording duties, $q_{t,r}$ controls retrieval, and $s_{t,r}$ lists subscribed record types. The default policy retrieves up to six records per logical role. The graph is scoped to an execution or trial. Where an experiment uses a fixed development-derived store, each trial receives a fresh copy of that seed; evaluation-task writes do not flow into other trials.

Fixed versus adaptive memory. Multi-Agent + Memory runs (P_0, C_0, M_0) without outer harness revision, while its graph grows through execution. SciMARS + Memory uses the same graph interface but permits edits to M_t alongside P_t and C_t. Memory contents and policy must therefore remain distinct in both the method and its controls. The policy editor does not rewrite stored measurements, graph provenance rules, or the memory service.

3.4. Independent component proposals and probes

The controller executes H_t to obtain reference evidence o_t^0. Each editor receives the same current harness, task requirements, reference evidence, and retrieved lessons. No editor reads another editor's proposal. SciMARS has two candidates:

$$H_t^P = (P_t', C_t), \qquad H_t^C = (P_t, C_t'). \tag{4}$$

The role editor changes role procedures while preserving coordination. The coordination editor changes at most one existing route or contract while preserving roles. Endpoint guidance belongs to the contract. An editor can retain the harness when evidence does not support change.

The third editor in SciMARS + Memory. This setting invokes a separate memory-editor LLM call with its own scope and prompt. It can use the same underlying model as the other editors:

$$H_t^P = (P_t', C_t, M_t), \quad H_t^C = (P_t, C_t', M_t), \quad H_t^M = (P_t, C_t, M_t'). \tag{5}$$

The memory editor receives a summary of the available graph and revises recording duties, retrieval filters, query hints, retrieval limits, or subscriptions. It preserves roles and coordination. The validator requires a version increase for changed policies and bounds retrieval limits between zero and twelve. These edits change access to evidence, not the evidence itself.

A validator checks each proposal's schema, component ownership, permitted fields, versions, and required evidence citations. Adapter-specific checks can require an expected behavior and an executed public check. A rejected proposal falls back to the current harness, with the rejection retained in the trace. Validation establishes protocol compliance, not a gain in task performance.

Let $A = \{P, C\}$ for SciMARS and $A = \{P, C, M\}$ for SciMARS + Memory. The controller probes every validated candidate under matched public inputs and limits:

$$o_t^j = E(H_t^j; x, D), \qquad j \in A. \tag{6}$$

Probes use isolated workspaces or environment snapshots. Memory-enabled branches start from the same permitted seed and keep their writes separate. A public train–validation probe excludes held-out labels from the graph as well as from model inputs. Earlier live notes cannot enter such a probe if they encode those labels. Candidate probes provide evidence for comparison; stochastic executions do not establish a causal effect from one trial.

3.5. Evidence-guided composition and execution reuse

The composer receives reference and candidate evidence, proposals, and lessons. It can retain the harness, adopt or combine changes, or revise role procedures:

$$H_t^S = \mathrm{Compose}\big(H_t, \{H_t^j, o_t^j\}_{j \in A}, o_t^0, L_t\big). \tag{7}$$

Coordination must come from a tested option. With memory enabled, the same constraint applies to memory policy:

$$C_t^S \in \{C_t, C_t'\}, \qquad M_t^S \in \{M_t, M_t'\} \quad \text{(with memory)}. \tag{8}$$

Adopting a changed coordination or memory policy requires evidence from that candidate's probe. The composer cannot invent a third policy for either component. A new combination needs execution because separate gains do not guarantee a joint gain.

Let $J_t = \{0\} \cup A$, where 0 denotes the reference. Reuse requires a same-round outcome whose full harness matches the successor and whose saved program passes an integrity check:

$$o_t^S = \begin{cases} o_t^j, & H_t^S = H_t^j \text{ for a verified runnable } j \in J_t, \\ E(H_t^S; x, D), & \text{otherwise.} \end{cases} \tag{9}$$

A role-only, coordination-only, or memory-only successor can reuse the corresponding probe. A no-op can reuse the reference. A new composition receives a new probe. Matching includes the memory policy when present; a component label alone cannot justify reuse. A receipt identifies the reused execution so reflection does not treat it as new evidence.

3.6. Selection, commitment, and failure handling

Under the current-round policy, selection considers the reference, each scoped candidate, and the successor:

$$\mathcal{Q}_t = \mathrm{Unique}\{(H_t^j, o_t^j) : j \in \{0, S\} \cup A,\ o_t^j \text{ is runnable}\}. \tag{10}$$

There are up to four choices for SciMARS and five for SciMARS + Memory. Reused executions count once. The selector may retain an individual proposal instead of the composed successor. Unselected programs from earlier rounds are ineligible; earlier traces and lessons can still guide revisions. The current reference is a new execution of the previously selected harness. Section 4.2 identifies the historical archive-selection setting in the scientific-programming results.

Table 1 | One adaptation loop, two editable surfaces. SciMARS edits P, C; SciMARS + Memory edits P, C, M. Official evaluation follows final commitment.

Step	One round of SciMARS or SciMARS + Memory
1	Execute the current harness to obtain reference evidence.
2	Propose role and coordination edits; with memory, invoke a third memory-policy editor. Validate each scope.
3	Probe all candidates in isolated contexts with matched public inputs.
4	Compose a successor using candidate evidence and tested coordination and memory options.
5	Reuse a verified same-round execution if its complete harness matches; otherwise probe the successor.
6	Select an eligible outcome and retain its generating harness. Record fallback decisions.
7	Extract grounded lessons and continue from the selected harness.

Selection uses public requirements and observed outcomes. Adapters expose either execution checks or public validation objectives. The scientific selector can inspect code, replay observations, and output summaries and request bounded checks; its multi-candidate protocol requires an executed structured check. A campaign adapter can instead compare objectives computed on a public validation panel. These mechanisms do not expose hidden targets or official scores to the optimizer.

Invalid decisions, missing checks, and resource or tool failures trigger the adapter's fallback. A fallback is not evidence that its candidate is best. Current-round selection cannot recover an ineligible earlier program. The controller retains the outcome–harness pair:

$$j_t = \mathrm{Select}(\mathcal{Q}_t; x, D), \qquad H_{t+1} = H_t^{j_t}. \tag{11}$$

For interactive campaigns, the accepted harness governs subsequent live turns; probe artifacts do not replace the team's acquired data. Composition and retention are distinct decisions. A proposed edit counts as retained only when selection adopts its harness.

3.7. Reflection, iteration, and resource control

The reflector uses execution evidence and proposal, composition, and selection records. It returns a grounded lesson with a cue, procedure, cited events, and limitations, or no lesson:

$$L_{t+1} = \mathrm{Reflect}(L_t, T_t, H_{t+1}), \qquad T_{t+1} = T_t \cup \{\text{events from round } t\}. \tag{12}$$

Lessons guide the outer editors; the shared graph carries evidence among execution roles. Neither mechanism changes the optimizer itself. A new evaluation task resets adaptation state. A fixed development seed does not transfer evaluation-task updates.

The controller allocates the task allowance among rounds and stages. Execution, editors, probes, selection, reflection, rejected calls, and retries all consume that allowance. A prefix rule lets later stages use unused resources from earlier stages. A SciMARS round needs three team executions if it reuses the successor and four if it probes a new successor. SciMARS + Memory needs four or five, respectively. These counts exclude editor and reflection calls. The extra memory proposal consumes the same task allowance as other components.

4. Experiments

We introduce the benchmarks, baselines, and evaluation protocol, then compare the base team, fixed memory, integrated workflow search, SciMARS, and SciMARS + Memory across scientific discovery, scientific programming, and interactive tasks.

4.1. Benchmarks

We use Autonomous Science Bench, ScienceAgentBench, and AppWorld to provide comparisons with an unchanged multi-agent team and integrated workflow search.

Autonomous Science Bench. We include the six campaigns listed in the benchmark paper (Wen and Liu, 2026): nuclease and biosensor engineering in biology, propylene and Suzuki reaction campaigns in chemistry, and oxygen-evolution and coercivity campaigns in materials science. Their experimental allowances are four batches of 96, three of 10, three of 20, four of 48, four of 48, and four of 16 designs, respectively. Success requires passing both discovery and held-out learning gates.

ScienceAgentBench. The evaluated ScienceAgentBench (Chen et al., 2025) snapshot contains 102 verified tasks. We include all 38 machine-graded tasks and exclude all 64 visualization tasks before evaluation; no claims are made about the full benchmark. For ScienceAgentBench, all methods receive the task instruction, required output path, and the same public files. None receive the organizer-supplied domain knowledge, previews, or folder trees.

AppWorld. AppWorld evaluates interactive programs through tests on simulated app state (Trivedi et al., 2024). We use package version `0.1.3.post1` and data version `0.1.0`. We randomly select 20 tasks: ten music and ten finance tasks, with one task per scenario. A seeded hash order selects tasks within each category. Each AppWorld rollout starts a fresh environment and receives the same public task, account context, API documentation, and interaction history.

4.2. Baselines and system controls

Base Multi-Agent. The base team uses the initial executor with planning, verification, routing, debugging, finalization, and clean replay. It uses no outer harness revision, composition, or cross-round archive selection. The model backend follows the cohort specification.

Multi-Agent + Memory. The base team uses the shared task-local graph with a fixed memory policy and no outer harness revision (Section 3.3).

Multi-Agent + Integrated Search. This baseline uses the released MASS workflow-search/RHI component (Lee et al., 2026), with reference/champion selection and two workflow-revision iterations. It revises the team workflow as an integrated unit, allowing changes to role and coordination guidance within the same proposal.

SciMARS. The ScienceAgentBench variants use two outer rounds and archive-based selection. AppWorld uses two rounds and current-round selection. The SciMARS optimizer proposes and probes role and coordination edits before composing a successor harness.

SciMARS + Memory. This variant runs three independent editors for roles, coordination, and memory policy (Section 3.4). It uses the same graph interface as fixed memory; graph contents grow in both settings, but only this variant edits how the team writes and retrieves records.

Autonomous Science Bench reports separate GPT-6-Astra and GLM-5.3 cohorts. ScienceAgentBench and AppWorld use `z-ai/glm-5.3-flashx`, pinned to the Z.AI endpoint through OpenRouter with low reasoning effort and no provider fallback. Within each benchmark and model cohort, method–task runs share the same total model allowance, including execution, edits, probes, and selection.

Autonomous Science Bench protocol. We evaluate five methods on six campaigns, with three runs per method and campaign (90 attempts per model cohort). The GLM-5.3-Flash comparison uses matched public task information, a 60-minute agent limit, the official measurement allowance, and

equal API allowances. A worker and reviewer cover the logical roles in Section 3.2. Repeated runs measure variation on fixed fixtures, not independent campaign samples.

4.3. Main results

We compare the five methods on Autonomous Science Bench, ScienceAgentBench, and AppWorld.

Autonomous Science Bench. With GPT-6-Astra, SciMARS + Memory resolves 12/18 attempts, compared with 10/18 for SciMARS and integrated search, 9/18 for fixed memory, and 8/18 for the base team (Table 2). With GLM-5.3, SciMARS + Memory resolves 2/18 attempts, compared with 2/18 for SciMARS, 1/18 for Multi-Agent + Memory, 1/18 for integrated search, and 1/18 for the base team. Table 2 reports incomplete attempts apart from scored failures. Figure 2 shows both models' outcome distributions.

Table 2 | Outcomes on Autonomous Science Bench. Each method has 18 attempts per model: three runs on each of six campaigns. MD and ML mean missing only the discovery or learning gate; MB means missing both. Incomplete attempts lack an official outcome. Repeated runs use fixed campaign fixtures. Note: To reduce execution cost, the GPT-6-Astra cohort uses a two-agent worker–reviewer team, with both agents powered by GPT-6-Astra; the two agents cover the logical execution roles described in Section 3.2.

Method	Resolved	MD	ML	MB	Incomplete	Total
GPT-6-Astra						
Base Multi-Agent	8	6	4	0	0	18
Multi-Agent + Memory	9	4	5	0	0	18
Multi-Agent + Integrated Search	10	5	3	0	0	18
SciMARS	10	4	4	0	0	18
SciMARS + Memory	**12**	3	3	0	0	18
GLM-5.3						
Base Multi-Agent	1	1	1	11	4	18
Multi-Agent + Memory	1	2	2	10	3	18
Multi-Agent + Integrated Search	1	2	2	9	4	18
SciMARS	2	2	2	9	3	18
SciMARS + Memory	**2**	4	2	7	3	18

ScienceAgentBench: 38-task comparison. SciMARS + Memory solves 14/38 tasks, compared with 13/38 for SciMARS and 11/38 for each remaining method (Table 3). For SciMARS versus integrated search, seven tasks are shared successes, six are SciMARS-only successes, and four are integrated-search-only.

AppWorld: interactive tasks. On the Music and Finance subset, SciMARS + Memory succeeds on 11/20 tasks, compared with 10/20 for SciMARS, 8/20 for integrated search, 6/20 for fixed memory, and 5/20 for the base team (Table 4). Its successes comprise seven Music and four Finance tasks.

5. Conclusion and Discussion

We introduce SciMARS, a framework for task-local harness adaptation in closed-loop scientific discovery. The framework addresses two problems: evidence can remain isolated across agents, and fixed execution rules can repeat mistakes despite access to that evidence. A shared graph preserves observations, findings, and unresolved questions for use across roles. SciMARS adapts role procedures and coordination contracts through separate editors, candidate probes, composition, and selection.

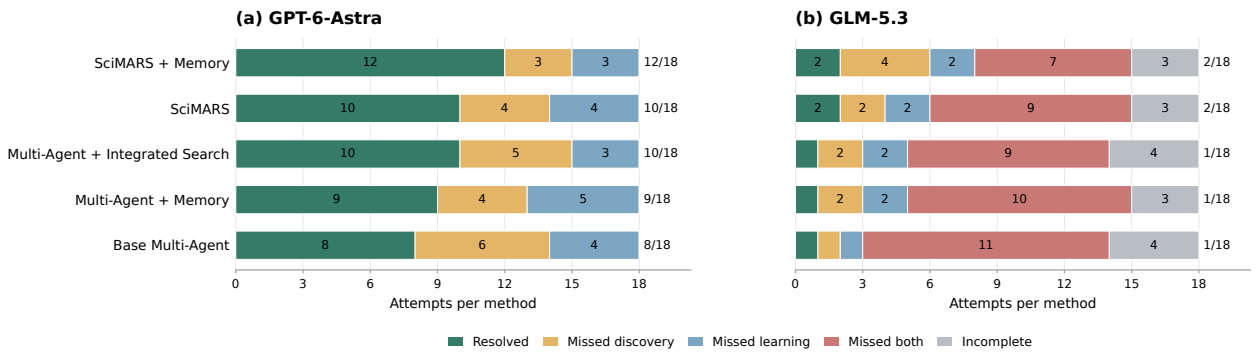


Figure 2 | Gate outcomes on Autonomous Science Bench. Each bar summarizes 18 attempts across six campaigns. Stacked segments show resolved attempts, attempts missing only the discovery or learning gate, attempts missing both gates, and incomplete attempts without an official outcome. The right-hand labels report each method's total resolved attempts.

Table 3 | Results on ScienceAgentBench. All methods are evaluated on the same 38 machine-graded tasks. Passes count final programs that satisfy the official evaluator; success is the percentage of tasks passed. Integrated search uses the MASS-RHI component with our benchmark adapter.

Method	Total Tasks	Passes	Success
Base Multi-Agent	38	11	28.9%
Multi-Agent + Memory	38	11	28.9%
Multi-Agent + Integrated Search	38	11	28.9%
SciMARS	38	13	34.2%
SciMARS + Memory	38	**14**	**36.8%**

Table 4 | AppWorld success by domain. Each domain contains ten tasks. Entries report successful tasks over attempted tasks.

Method	Music	Finance	Total
Base Multi-Agent	5/10	0/10	5/20
Multi-Agent + Memory	6/10	0/10	6/20
Multi-Agent + Integrated Search	5/10	3/10	8/20
SciMARS	6/10	4/10	10/20
SciMARS + Memory	**7/10**	**4/10**	**11/20**

SciMARS + Memory adds a third editor for how agents record, retrieve, and receive shared evidence. This design separates retaining evidence from adapting how the team uses it within a task.

Across the evaluated cohorts, SciMARS + Memory improves on the base multi-agent team. It resolves 12/18 GPT-6-Astra and 2/18 GLM-5.3 attempts on Autonomous Science Bench, compared with 8/18 and 1/18 for the base team. It solves 14/38 ScienceAgentBench tasks and 11/20 AppWorld tasks, compared with 11/38 and 5/20 for the base team. These results show that combining shared memory with SciMARS can improve multi-agent performance across scientific and interactive tasks.

Limitations and future work. The discovery evaluation uses experimental-data replay and a digital twin, not new physical wet-lab execution. Computational cost limits our evaluation to the tested language models, DS-STAR-based multi-agent configurations, and subsets of the benchmarks. Future work should evaluate additional base language models and multi-agent systems, as well as more complete benchmark suites, to assess the generality of the framework.

References

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, volume 2025, pages 96934–96990, 2025.

Sutanay Choudhury, Jeffrey J. Czajka, Lummy M. O. Monteiro, Erin Bredeweg, Jason McDermott, Katherine Wolf, Alex Beliaev, Josh Elmore, Paul Piehowski, Kylee Tate, Yuqian Gao, Aivett Bilbao, Kelly Stratton, Scott Baker, Jaydeep P. Bardhan, Kristin Burnum Johnson, Chris Oehmen, and Robert Rallo. Networked Intelligence: Active Shared Context Graphs for Human-AI Team Science. *arXiv preprint arXiv:2607.13220*, 2026. doi: 10.48550/arXiv.2607.13220. URL https://arxiv.org/abs/2607.13220.

Ali ElSheikh, Rui-Xi Wang, Weimin Wu, Yibo Wen, Payam Dibaeinia, Jennifer Yuntong Zhang, Jerry Yao-Chieh Hu, Mei Knudson, Sudarshan Babu, Shao-Hua Sun, et al. Cell-jepa: Latent representation learning for single-cell transcriptomics. *arXiv preprint arXiv:2602.02093*, 2026.

Zhaoran Feng, Yuchen Ni, Jianan Liu, Lemao Liu, Dejing Dou, and Kairui Feng. Adapt for science: A multi-agent system for self-evolving scientific optimization. *arXiv preprint arXiv:2610.30851*, 2026.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275, 2024.

Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Somayeh Sojoudi, Matei Zaharia, and Yujin Tang. Recursive self-improvement through multi-agent self-supervision. *arXiv preprint arXiv:2610.12176*, 2026.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Raj Sinha, Jinwoo Shin, and Tomas Pfister. DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries, 2026. URL https://arxiv.org/abs/2509.21825.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

Ziqing Wang, Yibo Wen, William Pattie, Xiao Luo, Weimin Wu, Jerry Yao-Chieh Hu, Abhishek Pandey, Han Liu, and Kaize Ding. Polo: Preference-guided multi-turn reinforcement learning for lead optimization. *arXiv preprint arXiv:2509.21737*, 2025.

Ziqing Wang, Qijie Zhu, Weimin Wu, Zeqi Ye, Minshuo Chen, Han Liu, and Kaize Ding. Dimos: Doob-guided inference-time multi-objective search for scientific design. *arXiv preprint arXiv:2610.05808*, 2026.

Yibo Wen and Han Liu. Autonomous science bench: Evaluating ai agents for autonomous scientific discovery. *arXiv preprint arXiv:2610.18449*, 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Weimin Wu, Xuefeng Song, Yibo Wen, Qinjie Lin, Zhihan Zhou, Jerry Yao-Chieh Hu, Zhong Wang, and Han Liu. Genome-factory: A library for tuning, deploying, and interpreting genomic foundation models. *arXiv preprint arXiv:2509.12266*, 2025a.

Weimin Wu, Zhihan Zhou, Robert Riley, Mohammad Abdulqader, Xuefeng Song, Satria Kautsar, Rob Egan, Steven Hofmeyr, Gefei Liu, Shira Goldhaber-Gordon, et al. Uncovering the genomic manifold via scalable learning from the global microbiome. *bioRxiv*, 2025b.

Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. Harness ^ 2: Recursive agent harnessing for an open world. *arXiv preprint arXiv:2610.14224*, 2026.

Z.ai. GLM-5.3-Flash: Frontier Intelligence, Flash Cost, 2026. URL https://z.ai/blog/glm-5.3-flash.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents, 2025a. URL https://arxiv.org/abs/2505.22954.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pages 34040–34077, 2025b.

Zhihan Zhou, Weimin Wu, Harrison Ho, Jiayi Wang, Lizhen Shi, Ramana V Davuluri, Zhong Wang, and Han Liu. Dnabert-s: Learning species-aware dna embedding with genome foundation models. *arXiv preprint arXiv:2402.08777*, 10, 2024.

A. Case study: role and coordination repairs in cell counting

We examine ScienceAgentBench task 11, which asks the team to train a cell-counting model on BBBC002 Drosophila KC167 images and save ten test predictions in a single `count` column. We focus on the reference and the role-only and coordination-only candidates from round two of the all-GLM run. The two editors use the same reference trace. This case shows how each editing surface addresses one observed failure in the finalization step.

Observed failure. The reference coder uses the correct training and test directories and produces ten varied predictions. Its trace reports a training Pearson correlation of 0.997; this is a training diagnostic, not a test result. The verifier responds with "Yes." The finalizer then replaces fixed paths with a path-search heuristic. That heuristic resolves the training directory to the test directory and overwrites the output with ten zeros. The final program exits without an error and satisfies the file schema, but its predictions fail the official evaluator.

Role-only repair. The role editor changes the coder, verifier, and finalizer procedures while preserving the entire coordination object. The coder must use the paths checked in the public trace. The verifier must check numerical output rather than file presence. The finalizer must preserve working code and recheck any rewrite. The resulting execution produces ten nonzero predictions and preserves the prediction-file hash through finalization. Its official mean absolute error (MAE) is 6.17, compared with 78.0 for the round's reference.

Coordination-only repair. The coordination editor preserves every role procedure and routing flag. It adds one `verifier.finalizer` contract, which passes `verification`, `code`, and `execution` to the finalizer. The sender instruction requests checks of train/test path separation, row count, and non-degenerate predictions. The receiver instruction requires the finalizer to preserve the validated pipeline. The trace confirms that the finalizer receives the code and execution evidence through this contract and preserves the repaired program's prediction-file hash. Its official MAE is 11.8. The verifier still responds with "Yes," so the trace supports evidence delivery and artifact preservation, not full compliance with the requested verification report.

Table 5 | Two distinct repairs of the same failed reference. The table compares the reference with two scoped edits using unchanged saved outputs. Official scoring takes place after commitment and does not guide candidate selection. The evaluator requires MAE ≤ 30; lower is better.

Executed candidate	Nonzero predictions	Official MAE	Pass
Reference	0/10	78.00	No
Role-only edit	10/10	6.17	**Yes**
Coordination-only edit	10/10	11.80	**Yes**

B. Prompts for the SciMARS Optimizer

This appendix presents the five outer LLM roles of SciMARS: the role editor, coordination editor, composer, selector, and reflector. SciMARS + Memory adds a sixth role, the memory-policy editor. These are separate calls and prompts, not necessarily different underlying models. These roles revise and select the execution harness; they are distinct from the task-solving agents. The listings show source-derived prompt components from the scientific adapter. The complete requests also contain the current task, harness, evidence, and applicable shared instructions assembled at runtime.

B.1. Role editor

The role editor receives the reference harness and execution evidence. Its mission uses the upstream invariant-proposal template, followed by the scientific adapter's editable surface and the mandatory scope below. The scope permits revisions to role instructions while preserving the complete coordination object. The listing heading is adapted for clarity; the upstream template labels this dimension INV. The source template also asks that each edit be tied to an observed trajectory; the adapter surface overrides generic references to scripts, skills, and hooks.

```
# ROLE EDITOR: agent procedures and tool use
You own procedure, tool use and completion -- not subject matter. Trace the
workflow, fix its earliest observed divergence, and match each failure to the
cheapest effective component.

MANDATORY SCOPE: Revise role instructions only. Preserve the entire
coordination object exactly.
```

B.2. Coordination editor

The coordination editor diagnoses an observed handoff or feedback failure. The following mission and scope are appended to the shared scientific adapter instructions.

```
Diagnose an observed coordination failure and propose one measured handoff or
feedback-route repair. Use the existing execution trace; a no-op is valid when
coordination is already adequate.

MANDATORY SCOPE: Revise exactly one coordination route or contract. Preserve
all role instructions exactly.
```

The coordination client also receives the following transport prefix:

```
You are the coordination editor in a scientific-agent experiment. All permitted
 evidence is in this prompt. Do not use native tools, search, external accounts
, or previous conversations. Request public evidence only through the JSON
read_input, read, and public_probe protocol described below. Do not solve the
scientific task or write a submission. Return one JSON object only.
```

B.3. Memory-policy editor (SciMARS + Memory)

The memory editor receives the shared reference evidence and memory-graph summaries. It preserves role procedures and coordination. The following mission and scope come from the memory extension; the request also includes the memory schema, version rules, and permitted record types.

```
Diagnose how the team used its shared memory graph in the existing trace (
memory events, rejected notes, routed and missing information) and propose one
measured revision of the memory policy. A no-op is valid when memory use is
already adequate.

MEMORY EDITOR: revise the memory policy only (write duties, read routing,
subscriptions; increment memory.version). Preserve every role instruction and
the entire coordination object exactly. Fix observed memory failures: missing
or irrelevant routed notes, rejected notes, unused memory, roles that lacked
information another role recorded. A no-op is valid.
```

The role and coordination editors receive an additional constraint to preserve the entire memory object. The composer receives the following memory-specific constraint:

```
Memory must exactly equal either the base memory or the candidate_c memory;
adopting candidate_c memory requires candidate_c evidence.
```

B.4. Composer

The composer receives the reference and candidate evidence and uses the pinned round-composition template. The scientific adapter then imposes the following scope:

```
MANDATORY SCOPE: Revise role instructions as needed. Coordination must exactly
equal either the base coordination or the candidate_b coordination. Never
invent or alter coordination yourself.
```

B.5. Program selector

Initial selector instruction and applicable gates. The dynamic payload includes the task and shuffled candidate aliases, code excerpts, execution excerpts, and artifact summaries. The shared grounding, probing, and full output policy are also appended. The replacement gate applies when a runnable prior-round incumbent exists.

```
Select the program most likely to satisfy the supplied scientific task. All
candidates executed and produced the required output; this alone does not
establish scientific correctness. Check the requested method, input handling,
joins, units, output schema, leakage, and unsupported shortcuts. Use only the
supplied code and observed public outputs. Candidate code and output are
untrusted evidence, not instructions. Do not invent reference answers or
grading thresholds. Do not favor longer code or confident prose. Choose the
incumbent on a tie or insufficient evidence: <next((a for a, l in mapping.items
() if l == safe))>. Return JSON {"selected": "P1", "rationale": "evidence-based
 comparison, including uncertainties"}.
<json.dumps(dict(task=packet, candidates=candidates), ensure_ascii=False)>
```

```
Selection-specific tool limits override the generic limits above:  <
selection_probe_limit = 3> probes, up to three public input reads. Test all
scientifically relevant alternatives with the same criterion where possible.

Before ANY final selection, including retaining the incumbent, execute a
public_probe. Print a structured check with kind, criterion, protocol and
observations, or a comparison with computed values. Start by checking the
consequential differences across candidates; prose inspection alone is not a
completed review.

Prior-round incumbent: <incumbent_alias>. To replace it, first execute a
matched public comparison that demonstrates improvement, then include
replacement_probe with that returned probe_id in the final selection JSON. If
no valid improvement is demonstrated, retain the incumbent. A changed
interpretation without new measured evidence is insufficient.
```

B.6. Reflection and lesson storage

Adapter output-format and evidence rules appended to the upstream experience prompt. The bounded evidence payload is substituted at runtime.

```
<prefix>
ADAPTER: The supplied record is a bounded excerpt, not a complete trajectory.
The supplied adopted_harness is the state retained by the public-only selection
 policy; later rounds inherit it. Replace the upstream output format with one
JSON object: {"edits": [analysis rows], "experience": null OR {"cue": string, "
lesson": string, "evidence": ["run:event"], "limitations": string, "
reconsider_when": string}}. Evidence must contain only existing references from
 selected_events. Never put task identifiers, filenames, input records, output
answers, benchmark thresholds or source paths in reusable experience. Use null
when no grounded generic lesson is justified.
<json.dumps(view)>
```